

Guide de Référence : Les Collections en Java

En-tête de cours – Informatique

1. Vue d'ensemble

Le framework des collections de Java (`java.util`) fournit des structures de données dynamiques pour stocker et manipuler des groupes d'objets. Contrairement aux tableaux statiques, elles s'ajustent automatiquement en taille.

La hiérarchie repose sur trois interfaces fondamentales :

- **List** : Collection ordonnée qui autorise les doublons et l'accès par index (basé sur 0).
- **Set** : Collection qui ne contient aucun doublon (unicité garantie).
- **Map** : Structure associative de paires *clé-valeur*, où chaque clé est unique.

2. Implémentations Courantes

Les Listes (List)

- **ArrayList** : Tableau dynamique. Accès rapide par index $O(1)$, mais insertion/suppression lente au milieu $O(n)$.
- **LinkedList** : Liste doublement chaînée. Insertion/suppression rapide si le nœud est connu, mais accès séquentiel plus lent $O(n)$.

Les Ensembles (Set)

- **HashSet** : Basé sur une table de hachage. Très performant $O(1)$, mais aucun ordre garanti.
- **TreeSet** : Basé sur un arbre rouge-noir. Les éléments sont triés automatiquement par ordre naturel $O(\log n)$.

Les Associations (Map)

- **HashMap** : Clés-valeurs non ordonnées, accès en $O(1)$.
- **TreeMap** : Clés triées automatiquement par ordre naturel $O(\log n)$.

3. Exemples de Code

```
import java.util.*;

public class Main {
    public static void main(String[] args)
    {
        // 1. ArrayList (Liste ordonnée)
        List<String> fruits = new
            ArrayList<>();
        fruits.add("Pomme");
        fruits.add("Banane");
        fruits.add("Pomme"); // Doublon
                               accepte

        // Parcours avec for-each
        for(String f : fruits) {
            System.out.println(f);
        }

        // 2. HashSet (Unicité)
        Set<String> unique = new HashSet
            <>(fruits);
        System.out.println("Taille unique
            : " + unique.size()); // 2

        // 3. HashMap (Dictionnaire)
        Map<String, Integer> stock = new
            HashMap<>();
        stock.put("Pommes", 50);
        stock.put("Bananes", 30);

        // Accès et parcours des entrées
        System.out.println("Stock pommes :
            " + stock.get("Pommes"));

        for(Map.Entry<String, Integer>
            entry : stock.entrySet()) {
            System.out.println(entry.
                getKey() + " : " + entry.
                getValue());
        }
    }
}
```

4. Exercices Pratiques

Exercice 1 : Analyse de notes (List) Écrivez un programme qui initialise une `ArrayList<Double>` avec 5 notes (ex : 75.5, 90.0, 60.0, 85.0, 45.5). Le programme doit parcourir la liste pour afficher :

- La note maximale et minimale.
- La moyenne générale de la classe.

Exercice 2 : Élimination de doublons et tri (Set)

Partez d'une liste de prénoms contenant des doublons (*Alice, Bob, Charlie, Alice, David, Bob*).

- Utilisez un `HashSet` pour éliminer les doublons.
- Transférez ensuite les éléments dans un `TreeSet` pour les afficher automatiquement triés par ordre alphabétique.

Exercice 3 : Annuaire simple (Map) Créez une `HashMap<String, String>` simulant un répertoire téléphonique ($\text{Nom} \rightarrow \text{Numéro}$).

- Ajoutez 3 contacts.
- Recherchez et affichez le numéro d'un contact spécifique à l'aide de sa clé.
- Affichez l'ensemble du répertoire sous format `Nom : Numéro`.